Firjan SENAI

Específico Profissional 2

Desenvolvimento de Sistemas

• LINGUAGENS DE PROGRAMAÇÃO • APIs E FRAMEWORKS
•INTEGRAÇÃO DE SISTEMAS • CONTROLE DE VERSÃO



2026



Sumário

1. **Fundamentos do Desenvolvimento de Sistemas**
   Visão sistêmica, planejamento, organização do trabalho e metodologias de desenvolvimento
2. **Qualidade e Requisitos do Sistema**
   Qualidade, integridade, usabilidade, segurança da informação e requisitos não funcionais
3. **Linguagens de Programação**
   Ferramentas, boas práticas e desenvolvimento multiplataforma
4. **APIs, Bibliotecas e Frameworks**
   Construção de rotinas de software e integração de recursos
5. **Integração de Sistemas**
   Comunicação entre subsistemas, plataformas e serviços
6. **Padrões de Projeto**
   Design Patterns, arquitetura e organização do código
7. **Controle de Versão**
   Gerência de configuração, repositórios, versionamento e rastreabilidade
8. **Documentação de Sistemas**
   Documentação técnica, registro das atividades e organização das informações



12

Requisitos funcionais e Regras de negócio

Aula 45 a 48 - Desenvolvimento de Sistemas

Instrutor: Vitor Hugo Almeida de Sousa



12.1

Situação do empréstimo

Adicionando atributo para verificar se livro foi devolvido

Situação do empréstimo

Como identificar se o livro foi devolvido?

Um empréstimo não termina quando é cadastrado. Precisamos registrar quando o livro retorna à biblioteca. Para isso, adicionaremos a data da devolução ao Model Emprestimo.

No Emprestimo.java:

- acrescente o atributo
- Adicione também seus métodos de acesso

Ao cadastrar um empréstimo, dataDevolucao permanece null: o livro ainda não foi devolvido. Quando a devolução ocorrer, a aplicação preencherá esse atributo com a data correspondente.

Essa informação permite distinguir um empréstimo em andamento de outro já encerrado, sem apagar o histórico.

Emprestimo.java

```java
private LocalDate dataDevolucao;
```

```java
public LocalDate getDataDevolucao() {
    return dataDevolucao;
}

public void setDataDevolucao(LocalDate dataDevolucao) {
    this.dataDevolucao = dataDevolucao;
}
```

Fique ON



Reinicie a aplicação e execute no MySQL:
DESCRIBE emprestimo;

Com spring.jpa.hibernate.ddl-auto=update, confira se a nova coluna foi acrescentada à tabela. Os empréstimos cadastrados anteriormente deverão permanecer sem data de devolução.

```
5 •  describe emprestimo;
```

esult Grid | Filter Rows: | Export: | Wrap Cell

Field	Type	Null	Key	Default	Extra
id	int	NO	PRI	NULL	auto_increment
data_emprestimo	date	YES		NULL	
aluno_id	int	NO	MUL	NULL	
livro_id	int	NO	MUL	NULL	
data_devolucao	date	YES		NULL	

Devolução no Service

Preenchendo a data de um empréstimo existente

Para registrar uma devolução, o Service receberá o ID do empréstimo e procurará o registro no banco. Se ele existir e ainda estiver em andamento, a aplicação preencherá dataDevolucao com a data atual.

No EmprestimoService.java:

- Adicione o método

O método procura o empréstimo pelo ID. Se não existir, devolve um Optional vazio. Se dataDevolucao estiver vazia, registra a data e salva a alteração.

Se o livro já tiver sido devolvido, o método retorna o registro como está. Assim, repetir a requisição não substitui a data da primeira devolução.

EmprestimoService.java

```java
public Optional<Emprestimo> devolver(Integer id) {
    Optional<Emprestimo> encontrado = emprestimoRepository.findById(id);

    if (encontrado.isEmpty()) {
        return Optional.empty();
    }

    Emprestimo emprestimo = encontrado.get();

    if (emprestimo.getDataDevolucao() == null) {
        emprestimo.setDataDevolucao(LocalDate.now());
        emprestimoRepository.save(emprestimo);
    }

    return Optional.of(emprestimo);
}
```

Devolução no Controller

Atualizando um empréstimo pelo ID

O Service já sabe registrar a devolução. Agora, vamos disponibilizar uma rota para que o Postman solicite essa operação.

No EmprestimoController.java:

- Adicione a importação
- Acrescente o método

No Postman, selecione PATCH, use http://localhost:8080/emprestimos/1/devolucao e deixe o Body vazio. Nesse desenho, o cliente solicita a devolução e o servidor preenche a data atual. A regra no Service mantém a primeira data se a requisição for repetida.

EmprestimoController.java

```java
import org.springframework.web.bind.annotation.PatchMapping;
```

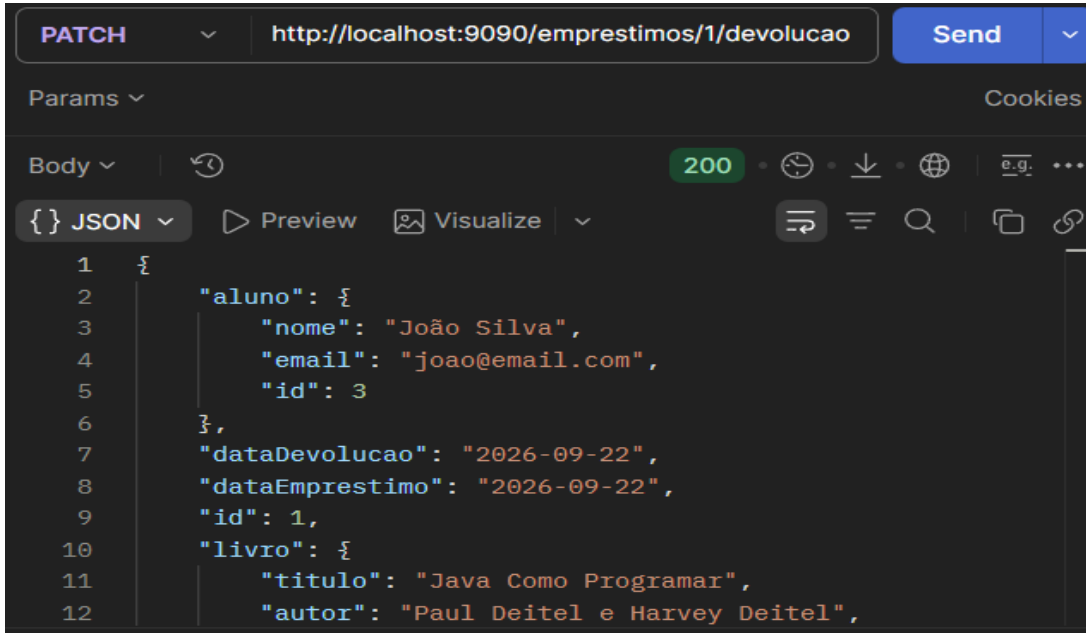
```java
@PatchMapping("/{id}/devolucao")
public ResponseEntity<Emprestimo> devolver(
        @PathVariable Integer id) {

    Optional<Emprestimo> emprestimo = emprestimoService.devolver(id);

    if (emprestimo.isPresent()) {
        return ResponseEntity.ok(emprestimo.get());
    }

    return ResponseEntity.notFound().build();
}
```

No Postman, selecione PATCH, use http://localhost:8080/emprestimos/1/devolucao



Devolvendo o livro

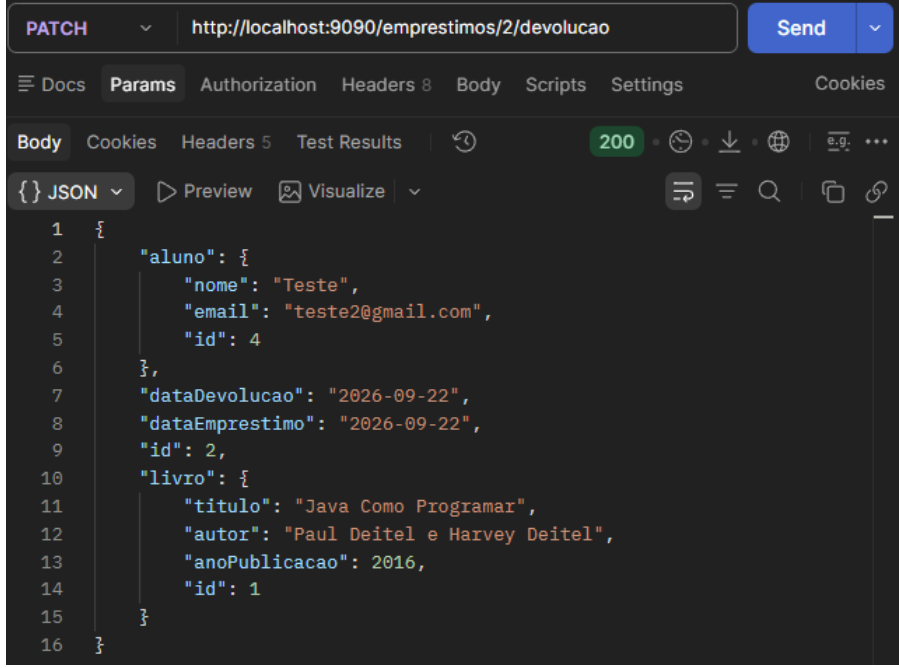
Tentativa de empréstimo

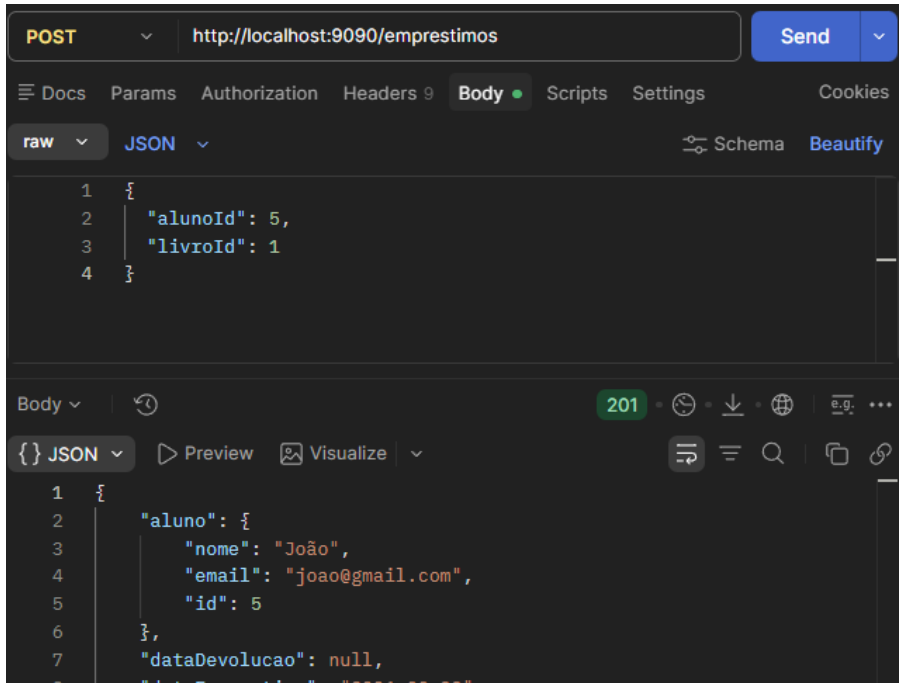
Com o servidor em execução, abra o Postman e confirme se a regra criada no EmprestimoService está funcionando.

Utilize a rota de devolução criada anteriormente:

- PATCH http://localhost:8080/emprestimos/1/devolucao

Depois da devolução, repita o cadastro do empréstimo. Como o empréstimo anterior agora possui uma data de devolução, o livro estará disponível novamente e poderá ser emprestado para outro aluno.





Consultando Empréstimos Ativos

Consultando livros que ainda estão emprestados

A rota GET /emprestimos apresenta todos os empréstimos, inclusive aqueles que já foram devolvidos.

Agora, criaremos uma consulta que retorne somente os empréstimos ativos, ou seja, aqueles cuja dataDevolucao ainda está vazia.

No arquivo EmprestimoRepository.java:

- adicione a importação
- acrescente o método

Entendendo o método

O Spring Data JPA cria a consulta automaticamente a partir do nome:

- findBy: solicita uma busca;
- DataDevolucao: indica o atributo utilizado na busca;
- IsNull: seleciona somente registros sem data de devolução.

EmprestimoRepository.java

src > main > java > br > com > senai > teste > repository > EmprestimoRepository.java

```java
package br.com.senai.teste.repository;

import java.util.List;

import org.springframework.data.jpa.repository.JpaRepository;

import br.com.senai.teste.model.Emprestimo;

public interface EmprestimoRepository
        extends JpaRepository<Emprestimo, Integer> {

    List<Emprestimo> findByDataDevolucaoIsNull();
}
```



12.2

Listando empréstimos ativos

Lista somente livros ainda emprestados

Listando Empréstimos Ativos

Criando o método no Service

Agora, o EmprestimoService utilizará a consulta criada no repositório.

Agora, criaremos uma consulta que retorne somente os empréstimos ativos, ou seja, aqueles cuja dataDevolucao ainda está vazia.

No arquivo EmprestimoService.java:

- adicione a importação
- acrescente o método

Criando o método no Controller

No arquivo EmprestimoController.java:

- adicione a importação
- acrescente o método

Nova rota

Para consultar os empréstimos ativos, utilize:

- GET http://localhost:8080/emprestimos/ativos

EmprestimoService.java

```java
public List<Emprestimo> listarAtivos() {
    return emprestimoRepository
            .findByDataDevolucaoIsNull();
}
```

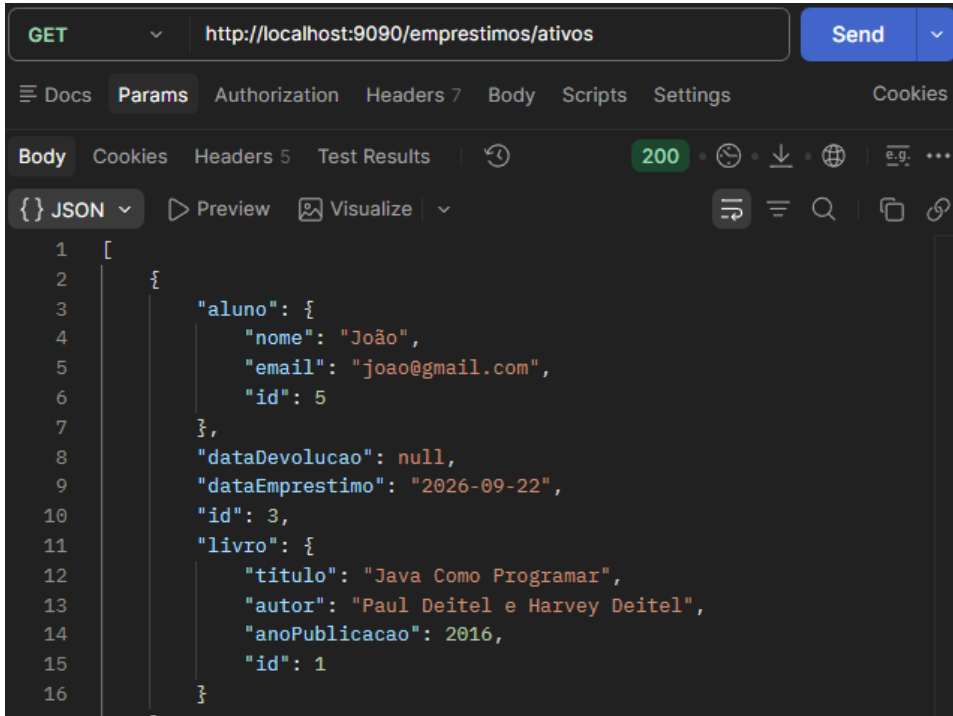
EmprestimoController.java

```java
@GetMapping("/ativos")
public ResponseEntity<List<Emprestimo>> listarAtivos() {

    List<Emprestimo> emprestimos = emprestimoService.listarAtivos();

    return ResponseEntity.ok(emprestimos);
}
```

No Postman





12.3

Validando cadastro de empréstimo

Impedindo empréstimo de livros sem data de devolução

Disponibilidade do Livro

Impedindo empréstimos duplicados

Atualmente, o sistema permite cadastrar mais de um empréstimo ativo para o mesmo livro.

Um empréstimo está ativo quando:

- possui um livro associado;
- o campo dataDevolucao ainda está vazio.

Para consultar essa informação, abra o arquivo EmprestimoRepository.java e acrescente o método:

```
boolean existsByLivroIdAndDataDevolucaoIsNull(Integer livroId);
```

O Spring Data JPA interpreta o nome do método e cria automaticamente a consulta ao banco de dados:

- existsBy: verifica se algum registro existe;
- LivroId: procura pelo identificador do livro;
- And: combina as condições;
- DataDevolucaoIsNull: verifica se a devolução ainda não foi registrada.

O resultado será:

- true: o livro já possui um empréstimo ativo;
- false: o livro está disponível.

EmprestimoRepository.java

src > main > java > br > com > senai > teste > repository > EmprestimoRepository.java

```
1  package br.com.senai.teste.repository;
2
3  import java.util.List;
4
5  import org.springframework.data.jpa.repository.JpaRepository;
6
7  import br.com.senai.teste.model.Emprestimo;
8
9  public interface EmprestimoRepository
10         extends JpaRepository<Emprestimo, Integer> {
11
   Turn into @Query | Go To Implementation | SELECT e.id FROM Emprestimo e WHERE e.livr
12     boolean existsByLivroIdAndDataDevolucaoIsNull(
13             Integer livroId);
14
15     List<Emprestimo> findByDataDevolucaoIsNull();
16 }
```

Verificando a Disponibilidade

Aplicando a regra no Service

Agora, o método criado no repositório será utilizado antes de salvar o empréstimo.

No arquivo EmprestimoService.java, :

- adicione as importações
- Dentro do método cadastrar, depois de verificar se o aluno e o livro existem, acrescente o método

Quando o livro já possuir um empréstimo ativo, a API responderá com: 409 Conflict

O código 409 indica que a requisição é válida, mas não pode ser concluída porque existe um conflito com o estado atual do livro.

EmprestimoService.java

```java
public Optional<Emprestimo> cadastrar(
        Integer alunoId, Integer livroId) {

    Optional<Aluno> aluno = alunoRepository.findById(alunoId);
    Optional<Livro> livro = livroRepository.findById(livroId);

    if (aluno.isEmpty() || livro.isEmpty()) {
        return Optional.empty();
    }

    boolean livroEmprestado = emprestimoRepository
            .existsByLivroIdAndDataDevolucaoIsNull(livroId);

    if (livroEmprestado) {
        throw new ResponseStatusException(
                HttpStatus.CONFLICT,
                "O livro já está emprestado");
    }

    Emprestimo emprestimo = new Emprestimo();
    emprestimo.setAluno(aluno.get());
    emprestimo.setLivro(livro.get());
    emprestimo.setDataEmprestimo(LocalDate.now());

    return Optional.of(emprestimoRepository.save(emprestimo));
}
```

Testando a Disponibilidade

Tentativa de empréstimo duplicado

Com o servidor em execução, abra o Postman e confirme se a regra criada no EmprestimoService está funcionando.

Primeiro, envie uma requisição:

- POST http://localhost:8080/emprestimos
- No campo Body, selecione raw, escolha o formato JSON e informe:

```
{
    "alunoId": 1,
    "livroId": 1
}
```

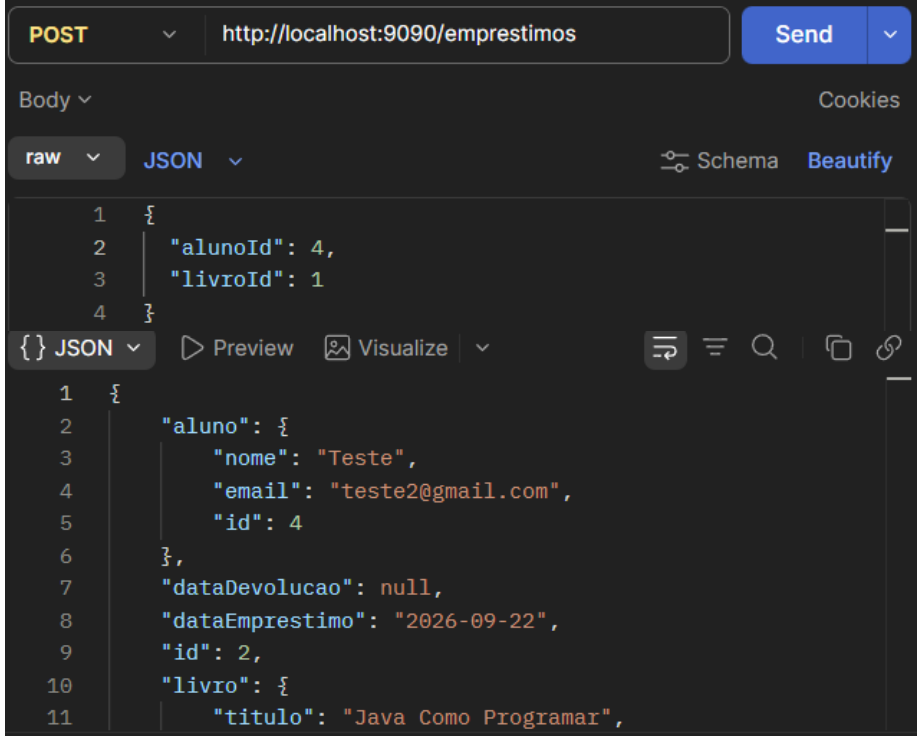
Se o aluno e o livro existirem e o livro estiver disponível, o servidor deverá responder: 201 Created

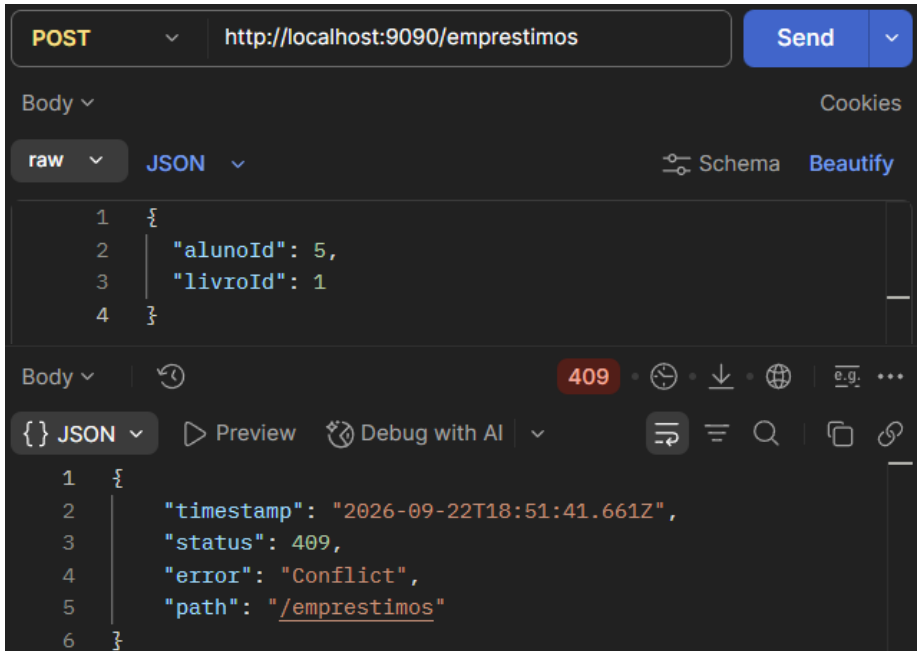
Repetindo a requisição:

- Sem registrar a devolução, envie novamente uma requisição usando o mesmo livrold:

```
{
    "alunoId": 2,
    "livroId": 1
}
```

Mesmo utilizando outro aluno, o servidor deverá responder: 409 Conflict







12.4

Histórico de empréstimo

Listando empréstimos de um aluno. Listando empréstimos de um livro específico

Histórico de Empréstimos

Consultando os empréstimos de um aluno

Até agora, o sistema permite listar todos os empréstimos ou somente os empréstimos ativos. Agora, criaremos uma consulta que retorna os empréstimos relacionados a um aluno específico.

No arquivo EmprestimoRepository.java:

- Adicione o método findByAlunoId

  O Spring Data JPA interpreta o nome do método:

  - findBy: realiza uma busca;
  - Aluno: acessa o aluno relacionado ao empréstimo;
  - Id: utiliza o identificador desse aluno como condição.

No arquivo EmprestimoService.java:

- adicione o método listarPorAluno

No arquivo EmprestimoController.java:

- adicione o método listarPorAluno

EmprestimoRepository.java

```java
List<Emprestimo> findByAlunoId(Integer alunoId);
```

EmprestimoService.java

```java
public List<Emprestimo> listarPorAluno(Integer alunoId) {
    return emprestimoRepository.findByAlunoId(alunoId);
}
```

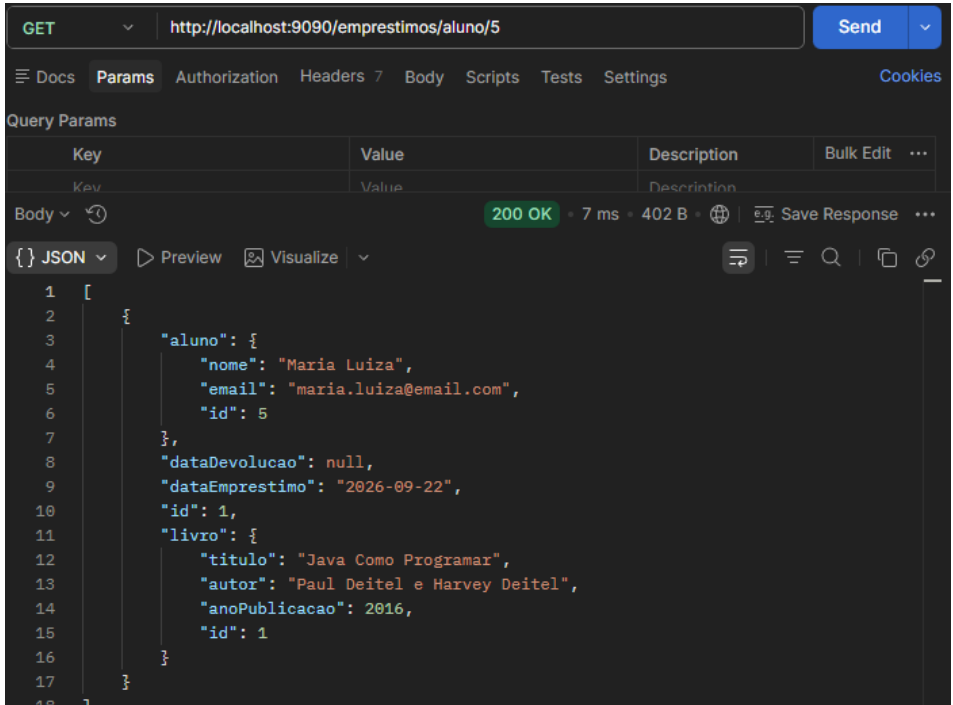
EmprestimoController.java

```java
@GetMapping("/aluno/{alunoId}")
public ResponseEntity<List<Emprestimo>> listarPorAluno(
        @PathVariable Integer alunoId) {

    List<Emprestimo> emprestimos = emprestimoService.listarPorAluno(alunoId);

    return ResponseEntity.ok(emprestimos);
}
```

No Postman



Histórico de empréstimos de um livro

Até aqui, a API permite consultar o histórico de empréstimos de um aluno específico. Agora, aplique o mesmo raciocínio para descobrir quais alunos já retiraram determinado livro. Utilize a consulta anterior como referência e teste a nova funcionalidade no Postman.

Desafio



Crie a rota GET /emprestimos/livro/{id} para retornar todos os empréstimos relacionados ao livro informado, incluindo os ativos e os já devolvidos. Faça as alterações no Repository, Service e Controller, seguindo a estrutura utilizada na consulta por aluno. No Postman, confira se todos os registros retornados pertencem ao livro solicitado. Quando não houver empréstimos, a resposta deverá ser uma lista vazia.



12.4

Previsão de devolução

Adicionando data prevista de devolução no cadastro de empréstimo

Previsão de Devolução

Adicionando uma nova data ao empréstimo

Até agora, o empréstimo registra apenas a data em que o livro foi retirado e, posteriormente, a data em que foi devolvido.

Agora, adicionaremos uma data prevista para a devolução. Diferentemente da dataEmprestimo, essa informação será escolhida e enviada pelo cliente no momento do cadastro.

No arquivo Emprestimo.java:

- Acrescente o atributo
- Adicione também os métodos de acesso

Data no DTO

O model Emprestimo já está preparado para armazenar a data prevista. Agora, o DTO precisa receber essa informação enviada pelo cliente.

No arquivo EmprestimoRequest.java:

- Acrescente a importação
- Adicione o atributo
- Adicione os métodos de acesso

Emprestimo.java

```java
private LocalDate dataPrevistaDevolucao;
```

```java
public LocalDate getDataPrevistaDevolucao() {
    return dataPrevistaDevolucao;
}

public void setDataPrevistaDevolucao(
        LocalDate dataPrevistaDevolucao) {

    this.dataPrevistaDevolucao = dataPrevistaDevolucao;
}
```

EmprestimoRequest.java

EmprestimoRequest.java

src > main > java > br > com > senai > teste > dto > EmprestimoRequest.java > EmprestimoRequest > setDataPrevistaDevolucao(LocalDate)

```java
package br.com.senai.teste.dto;

import java.time.LocalDate;

public class EmprestimoRequest {

    private Integer alunoId;
    private Integer livroId;
    private LocalDate dataPrevistaDevolucao;

    public EmprestimoRequest() {
    }

    public Integer getAlunoId() {
        return alunoId;
    }

    public void setAlunoId(Integer alunoId) {
        this.alunoId = alunoId;
    }

    public Integer getLivroId() {
        return livroId;
    }

    public void setLivroId(Integer livroId) {
        this.livroId = livroId;
    }

    public LocalDate getDataPrevistaDevolucao() {
        return dataPrevistaDevolucao;
    }

    public void setDataPrevistaDevolucao(
            LocalDate dataPrevistaDevolucao) {

        this.dataPrevistaDevolucao = dataPrevistaDevolucao;
    }

}
```

Enviando a Data ao Service

Atualizando o fluxo de cadastro

O DTO já recebe a data prevista, mas o Controller ainda envia somente os IDs para o Service. Agora, os dois arquivos precisam ser atualizados para transportar e salvar a nova informação.

No arquivo EmprestimoController.java:

- No método cadastrar, recupere a data recebida
- Atualize a verificação dos campos obrigatórios
- Envie a data ao Service

No arquivo EmprestimoService.java:

- Atualize os parâmetros do método
- Na criação do empréstimo, acrescente a data prevista
- Envie a data ao Service

EmprestimoService.java

```java
public Optional<Emprestimo> cadastrar(
        Integer alunoId,
        Integer livroId,
        LocalDate dataPrevistaDevolucao) {

    Optional<Aluno> aluno = alunoRepository.findById(alunoId);
    Optional<Livro> livro = livroRepository.findById(livroId);

    if (aluno.isEmpty() || livro.isEmpty()) {
        return Optional.empty();
    }

    boolean livroEmprestado = emprestimoRepository
            .existsByLivroIdAndDataDevolucaoIsNull(livroId);

    if (livroEmprestado) {
        throw new ResponseStatusException(
                HttpStatus.CONFLICT,
                "O livro já está emprestado");
    }

    Emprestimo emprestimo = new Emprestimo();
    emprestimo.setAluno(aluno.get());
    emprestimo.setLivro(livro.get());
    emprestimo.setDataEmprestimo(LocalDate.now());
    emprestimo.setDataPrevistaDevolucao(
            dataPrevistaDevolucao);

    return Optional.of(emprestimoRepository.save(emprestimo));
}
```

EmprestimoController.java

```java
@PostMapping
public ResponseEntity<Emprestimo> cadastrar(
        @RequestBody EmprestimoRequest dados) {

    Integer alunoId = dados.getAlunoId();
    Integer livroId = dados.getLivroId();

    LocalDate dataPrevistaDevolucao = dados.getDataPrevistaDevolucao();

    if (alunoId == null
            || livroId == null
            || dataPrevistaDevolucao == null) {

        return ResponseEntity.badRequest().build();
    }

    Optional<Emprestimo> emprestimo = emprestimoService.cadastrar(
            alunoId,
            livroId,
            dataPrevistaDevolucao);

    if (emprestimo.isEmpty()) {
        return ResponseEntity.notFound().build();
    }

    return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(emprestimo.get());
}
```

Testando a Data Prevista

Enviando uma data pelo Postman

Com a aplicação em execução, abra o Postman e crie uma requisição:

- POST http://localhost:8080/emprestimos

Em Body, selecione raw e escolha o formato JSON.

Envie os IDs de um aluno e de um livro existentes, além da data prevista:

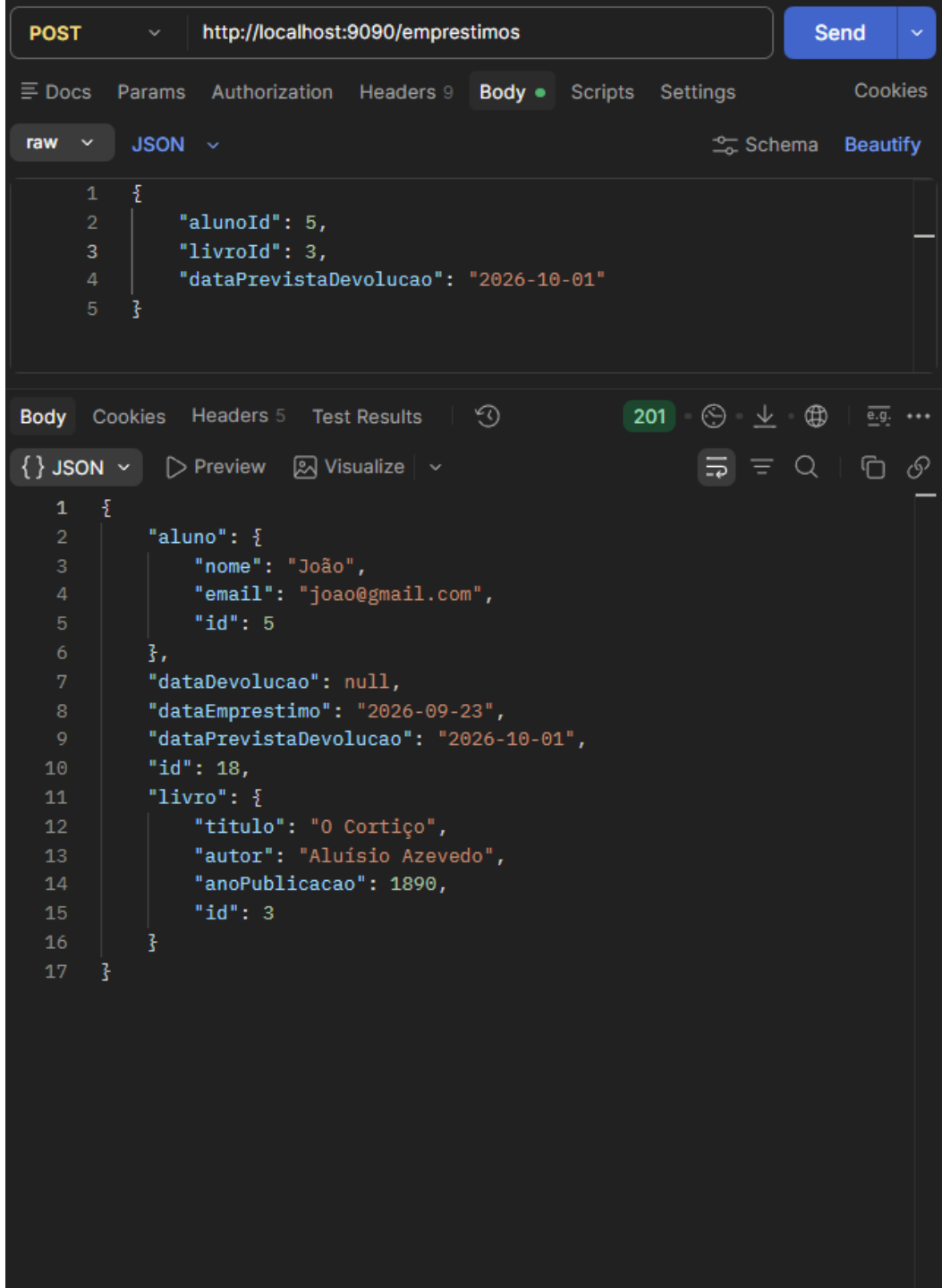
-
```
{
    "alunoId": 1,
    "livroId": 2,
    "dataPrevistaDevolucao": "2026-10-01"
}
```

A data deve seguir o padrão:

- AAAA-MM-DD

Nesse formato:

- AAAA representa o ano;
- MM representa o mês;
- DD representa o dia.



Seed

Utilize o seed para validar se os resultados das requisições estão vindo como esperado

Atenção



Acesse o arquivo DEV_SIS_45_48 (Spring Boot - RF e RN).sql. Caso não consiga o acesso solicite e execute todo o sql nele presente.

Validando a Data Prevista

Protegendo o cadastro do empréstimo

Agora, o DTO será responsável por verificar se os IDs foram informados e se a previsão de devolução está no futuro.

No arquivo EmprestimoRequest.java

- Adicione as importações
- Acrescente as anotações aos atributos

A anotação @NotNull impede que o campo seja omitido ou enviado com valor null.

A anotação @Future permite somente uma data posterior à data atual. Portanto, a data de hoje também será rejeitada.

EmprestimoRequest.java

src > main > java > br > com > senai > teste > dto > EmprestimoRequest.java > Empr

```java
package br.com.senai.teste.dto;

import java.time.LocalDate;

import jakarta.validation.constraints.Future;
import jakarta.validation.constraints.NotNull;

public class EmprestimoRequest {

    @NotNull (message = "O ID do aluno é obrigatório")
    private Integer alunoId;

    @NotNull(message = "O ID do livro é obrigatório")
    private Integer livroId;

    @NotNull(message = "A data prevista é obrigatória")
    @Future(message = "A data prevista deve estar no futuro")
    private LocalDate dataPrevistaDevolucao;

    public EmprestimoRequest() {
    }

    public Integer getAlunoId() {
        return alunoId;
    }

    public void setAlunoId(Integer alunoId) {
        this.alunoId = alunoId;
    }

    public Integer getLivroId() {
        return livroId;
    }

    public void setLivroId(Integer livroId) {
        this.livroId = livroId;
    }

    public LocalDate getDataPrevistaDevolucao() {
        return dataPrevistaDevolucao;
    }

    public void setDataPrevistaDevolucao(
            LocalDate dataPrevistaDevolucao) {

        this.dataPrevistaDevolucao = dataPrevistaDevolucao;
    }

}
```

Validando a Data Prevista

Ativando a validação no Controller

Agora, o DTO será responsável por verificar se os IDs foram informados e se a previsão de devolução está no futuro.

No arquivo EmprestimoController.java

- Adicione as importações
- Depois, coloque @Valid antes de @RequestBody

Com @Valid, o Spring verifica as anotações do DTO antes de executar o restante do método. A verificação manual pode ser removida.

EmprestimoController.java

```java
import jakarta.validation.Valid;
```

```java
@PostMapping
public ResponseEntity<Emprestimo> cadastrar(
        @Valid @RequestBody EmprestimoRequest dados) {

    Integer alunoId = dados.getAlunoId();
    Integer livroId = dados.getLivroId();
    LocalDate dataPrevistaDevolucao = dados.getDataPrevistaDevolucao();

    Optional<Emprestimo> emprestimo = emprestimoService.cadastrar(
            alunoId,
            livroId,
            dataPrevistaDevolucao);

    if (emprestimo.isEmpty()) {
        return ResponseEntity.notFound().build();
    }

    return ResponseEntity
            .status(HttpStatus.CREATED)
            .body(emprestimo.get());
}
```

Testando as Validações

Conferindo as regras no Postman

Com a aplicação em execução, crie uma requisição:

- POST http://localhost:8080/emprestimos

Teste 1 — Data não informada:

- Envie:
  -
```
{
    "alunoId": 1,
    "livroId": 2
}
```

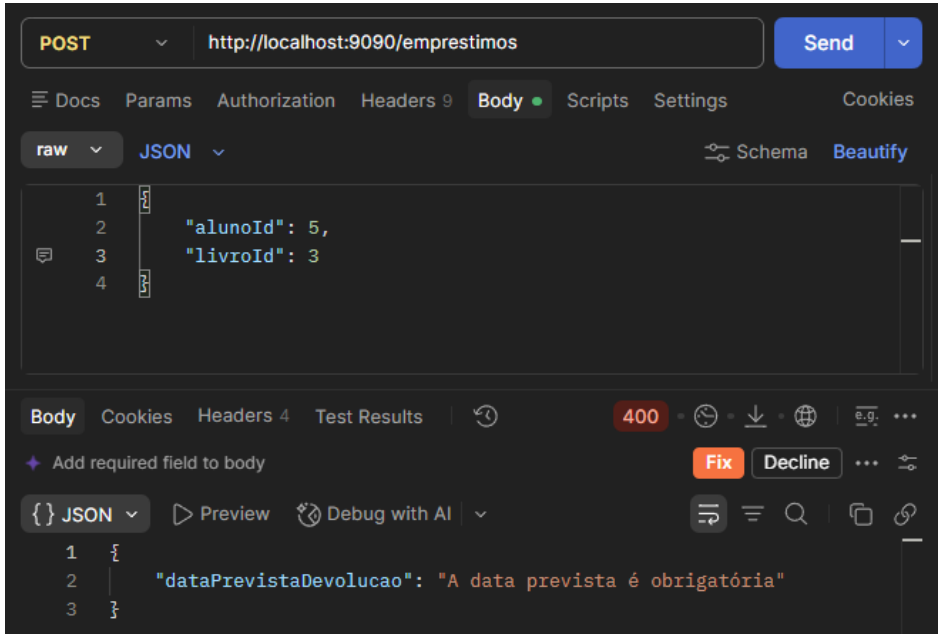
Resultado esperado: 400 Bad Request

Teste 2 — Data anterior:

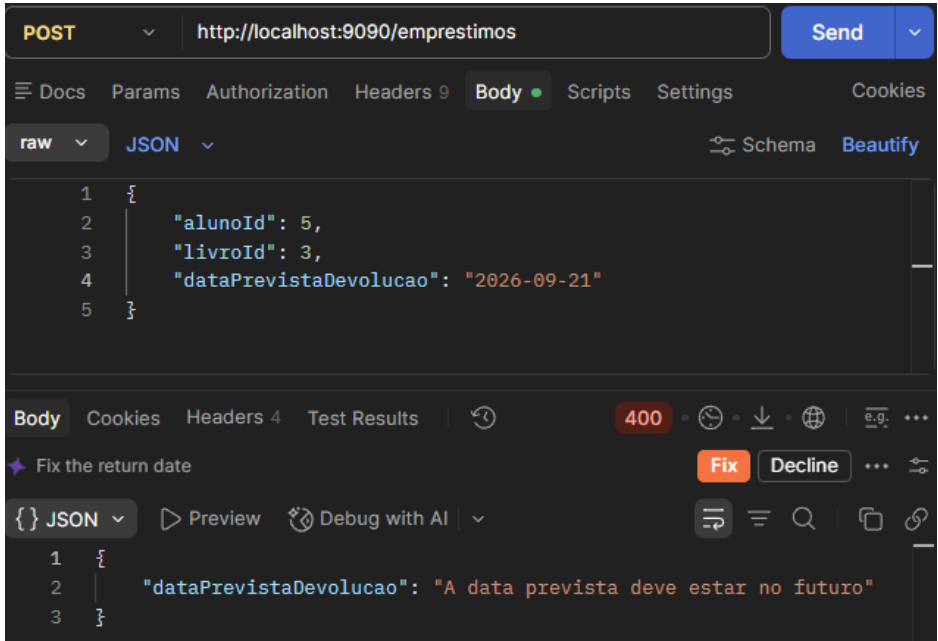
- Envie:
  -
```
{
    "alunoId": 1,
    "livroId": 2,
    "dataPrevistaDevolucao": "2026-09-20"
}
```

Resultado esperado: 400 Bad Request

Teste 1



Teste 2





12.5

Listando empréstimos atrasados

Listando os empréstimos com data prevista de devolução antes da data atual.

Empréstimos Atrasados

Consultando prazos que já venceram

Um empréstimo está atrasado quando a data prevista já passou e o livro ainda não foi devolvido.

No arquivo EmprestimoRepository.java:

- adicione a consulta por data prevista;
- considere somente registros com dataDevolucao vazia.

No arquivo EmprestimoService.java:

- crie o método listarAtrasados;
- utilize LocalDate.now() como data de comparação.

No arquivo EmprestimoController.java:

- crie a rota GET /emprestimos/atrasados.

EmprestimoRepository.java

```java
List<Emprestimo> findByDataPrevistaDevolucaoBeforeAndDataDevolucaoIsNull(
        LocalDate dataAtual);
```

EmprestimoService.java

```java
public List<Emprestimo> listarAtrasados() {
    return emprestimoRepository
            .findByDataPrevistaDevolucaoBeforeAndDataDevolucaoIsNull(
                    LocalDate.now());
}
```

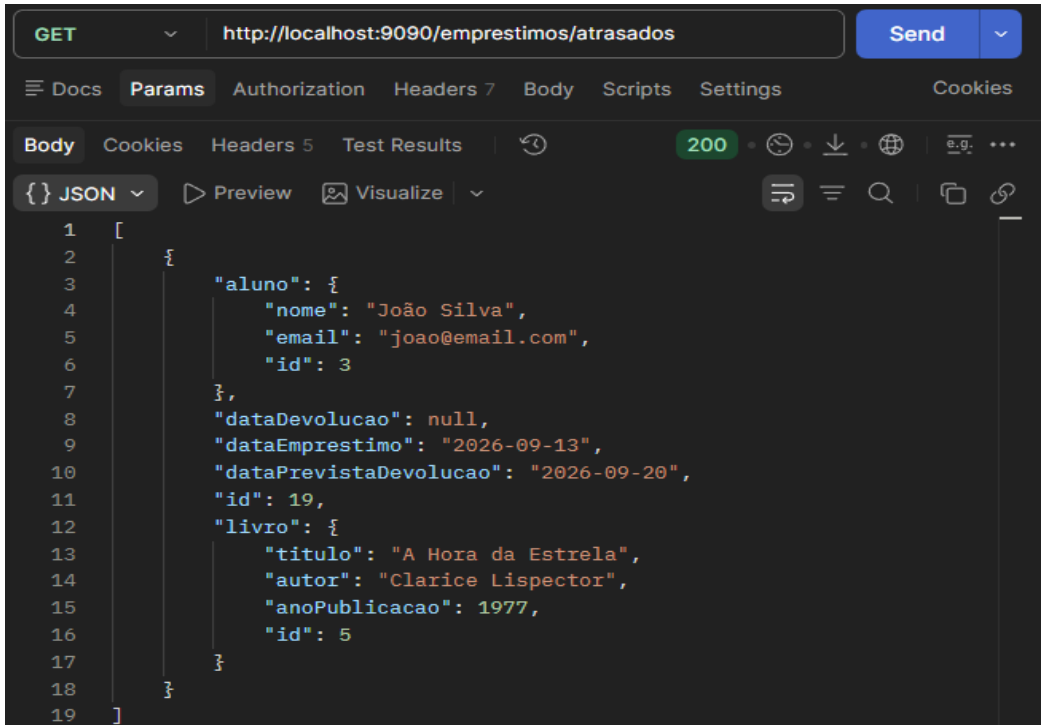
EmprestimoController.java

```java
@GetMapping("/atrasados")
public ResponseEntity<List<Emprestimo>> listarAtrasados() {

    List<Emprestimo> emprestimos = emprestimoService.listarAtrasados();

    return ResponseEntity.ok(emprestimos);
}
```

Postman



Empréstimos atrasados por aluno

A biblioteca já consegue consultar todos os empréstimos atrasados. Agora, precisa identificar as pendências de um aluno específico para orientar a devolução dos livros. Combine o que você aprendeu sobre o histórico por aluno com a consulta de prazos vencidos.

Desafio



Crie a rota GET /emprestimos/aluno/{id}/atrasados para retornar somente os empréstimos do aluno informado cuja data prevista seja anterior à data atual e que ainda não tenham sido devolvidos. Implemente a consulta no Repository e disponibilize-a pelo Service e Controller. No Postman, confira se empréstimos de outros alunos, já devolvidos ou com prazo ainda válido ficam fora do resultado.

Dica: o nome do método pode combinar condições usando And. Utilize como referência as consultas por aluno e por empréstimos atrasados.



12.6

Renovando empréstimos

Atualizando a data prevista de entrega

Firjan SENAI

Renovação do Empréstimo

Recebendo somente a nova data

Depois de identificar um empréstimo atrasado, o sistema poderá permitir a renovação do prazo. Nessa operação, somente a dataPrevistaDevolucao será alterada.

Não reutilizaremos o EmprestimoRequest, pois ele também exige alunoId e livroId. Criaremos um DTO específico para a renovação.

No pacote dto, crie:

- RenovacaoRequest.java

O JSON da renovação terá somente o campo necessário:

-
```
{
    "novaDataPrevista": "2026-10-15"
}
```

RenovacaoRequest.java

src > main > java > br > com > senai > teste > dto > RenovacaoRequest.java

```java
package br.com.senai.teste.dto;

import java.time.LocalDate;

import jakarta.validation.constraints.Future;
import jakarta.validation.constraints.NotNull;

public class RenovacaoRequest {

    @NotNull(message = "A nova data é obrigatória")
    @Future(message = "A nova data deve estar no futuro")
    private LocalDate novaDataPrevista;

    public RenovacaoRequest() {
    }

    public LocalDate getNovaDataPrevista() {
        return novaDataPrevista;
    }

    public void setNovaDataPrevista(
            LocalDate novaDataPrevista) {

        this.novaDataPrevista = novaDataPrevista;
    }
}
```

Renovação no Service

Alterando a data prevista

O Service receberá o ID do empréstimo e a nova data prevista. Antes de salvar a alteração, verificará se o empréstimo existe e se o livro ainda não foi devolvido.

No arquivo EmprestimoService.java:

- Adicione o método
- As importações necessárias provavelmente já existem no Service

Resultados possíveis:

- Empréstimo inexistente: retorna um Optional vazio;
- empréstimo já devolvido: responde 409 Conflict;
- empréstimo ativo: atualiza a data prevista e salva a alteração.

EmprestimoService.java

```java
public Optional<Emprestimo> renovar(
        Integer id,
        LocalDate novaDataPrevista) {

    Optional<Emprestimo> emprestimoOptional = emprestimoRepository.findById(id);

    if (emprestimoOptional.isEmpty()) {
        return Optional.empty();
    }

    Emprestimo emprestimo = emprestimoOptional.get();

    if (emprestimo.getDataDevolucao() != null) {
        throw new ResponseStatusException(
                HttpStatus.CONFLICT,
                "Um empréstimo devolvido não pode ser renovado");
    }

    emprestimo.setDataPrevistaDevolucao(
            novaDataPrevista);

    return Optional.of(
            emprestimoRepository.save(emprestimo));
}
```

Renovação no Controller

Criando a rota e testando a alteração

O Service já consegue renovar a data prevista. Agora, criaremos uma rota PATCH, pois somente uma parte do empréstimo será alterada.

No arquivo EmprestimoController.java:

- Adicione o método
- Adicione a importação de RenovacaoRequest

EmprestimoController.java

```java
@PatchMapping("/{id}/renovacao")
public ResponseEntity<Emprestimo> renovar(
        @PathVariable Integer id,
        @Valid @RequestBody RenovacaoRequest dados) {

    Optional<Emprestimo> emprestimo = emprestimoService.renovar(
            id,
            dados.getNovaDataPrevista());

    if (emprestimo.isEmpty()) {
        return ResponseEntity.notFound().build();
    }

    return ResponseEntity.ok(emprestimo.get());
}
```

Firjan SENAI

Postman

Teste no Postman

Selecione o método PATCH e informe o ID do empréstimo:

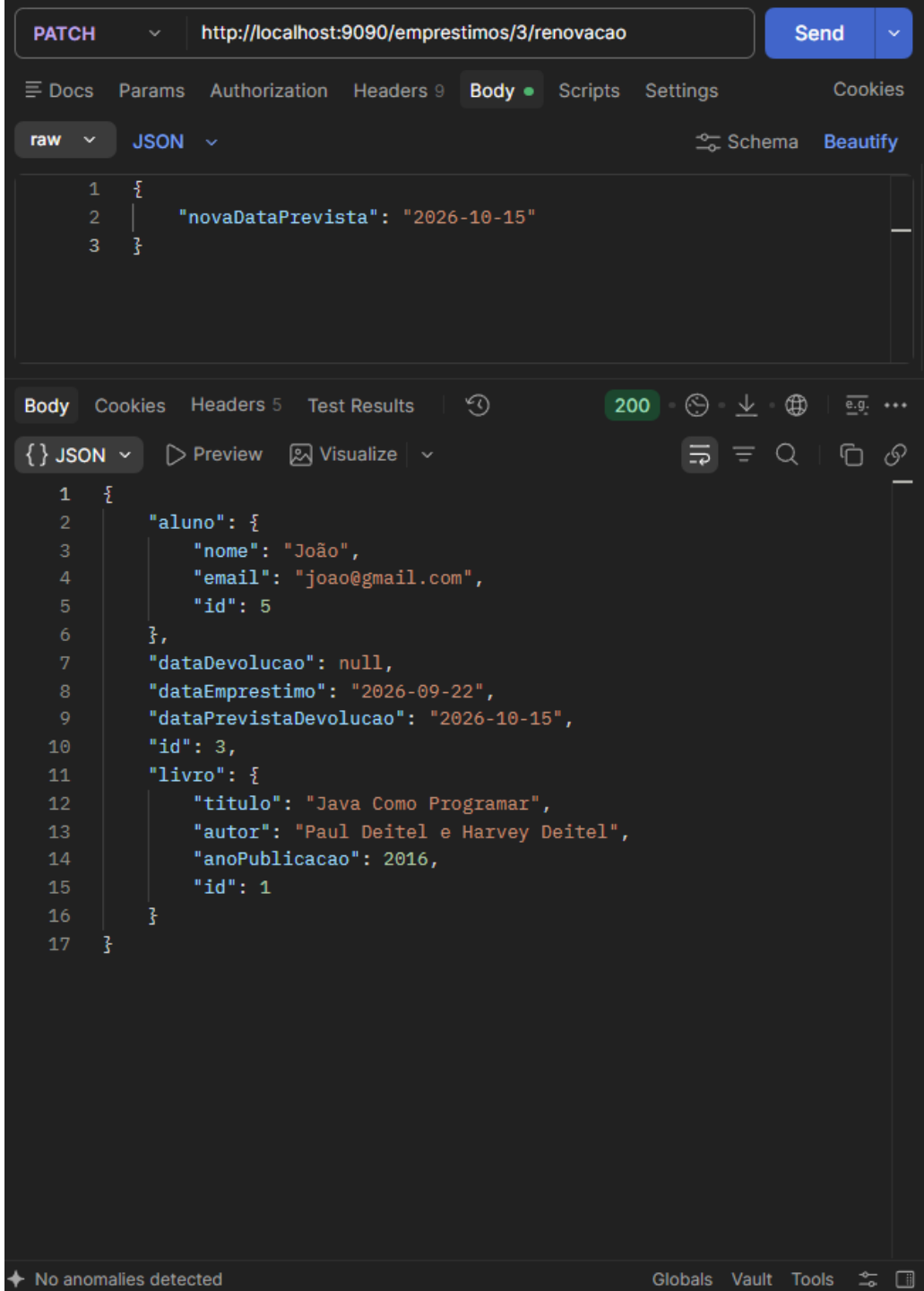
- http://localhost:8080/emprestimos/2/renovacao

Em Body → raw → JSON, envie:

-
```
{
    "novaDataPrevista": "2026-10-15"
}
```

Resultados esperados

- Empréstimo ativo: 200 OK com a nova data;
- ID inexistente: 404 Not Found;
- empréstimo já devolvido: 409 Conflict;
- data vazia ou que não esteja no futuro: 400 Bad Request.



Firjan SENAI

Validando a Renovação

A nova data deve ampliar o prazo

A anotação @Future garante que a nova data esteja no futuro. Porém, ela não compara essa data com a previsão que já está registrada.

Para evitar isso, no método renovar do EmprestimoService, adicione a verificação antes do setDataPrevistaDevolucao

Considere um empréstimo com previsão para:

- 20/10/2026

A data abaixo está no futuro, mas reduziria o prazo:

- 10/10/2026

```
public Optional<Emprestimo> renovar(
        Integer id,
        LocalDate novaDataPrevista) {

    Optional<Emprestimo> emprestimoOptional = emprestimoRepository.findById(id);

    if (emprestimoOptional.isEmpty()) {
        return Optional.empty();
    }

    Emprestimo emprestimo = emprestimoOptional.get();

    if (emprestimo.getDataDevolucao() != null) {
        throw new ResponseStatusException(
                HttpStatus.CONFLICT,
                "Um empréstimo devolvido não pode ser renovado");
    }

    if (!novaDataPrevista.isAfter(
            emprestimo.getDataPrevistaDevolucao())) {

        throw new ResponseStatusException(
                HttpStatus.CONFLICT,
                "A nova data deve ser posterior ao prazo atual");
    }

    emprestimo.setDataPrevistaDevolucao(
            novaDataPrevista);

    return Optional.of(
            emprestimoRepository.save(emprestimo));
}
```



12.7

Situação do empréstimo

Categorizando as situações de empréstimo

Situação do Empréstimo

Criando uma informação calculada

A situação do empréstimo pode ser determinada pelas datas já existentes. Não será necessário criar outro atributo nem uma nova coluna no banco de dados.

No arquivo Emprestimo.java:

- Adicione o método getSituacao()

Como a situação é definida?

- DEVOLVIDO: existe uma data de devolução;
- SEM_PREVISAO: registro antigo sem data prevista;
- ATRASADO: a previsão já passou e não houve devolução;
- ATIVO: o livro ainda não foi devolvido e o prazo não venceu.

Emprestimo.java

```java
public String getSituacao() {

    if (dataDevolucao != null) {
        return "DEVOLVIDO";
    }

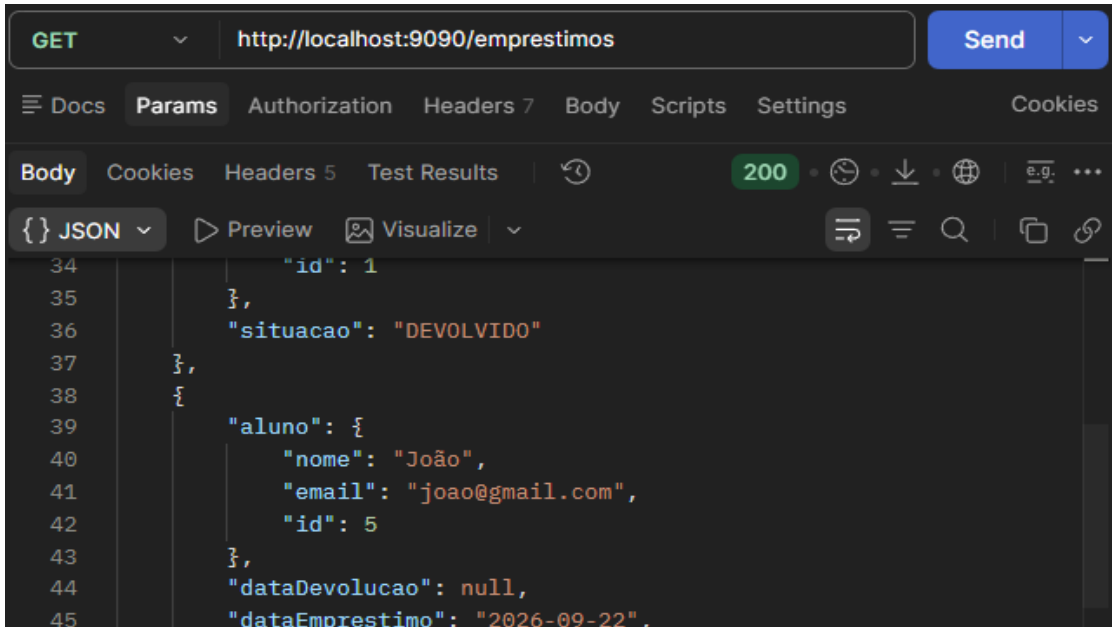
    if (dataPrevistaDevolucao == null) {
        return "SEM_PREVISAO";
    }

    if (dataPrevistaDevolucao.isBefore(
            LocalDate.now())) {

        return "ATRASADO";
    }

    return "ATIVO";
}
```

No Postman





12.8

Dias de atraso

Calculando dias de atraso

Calculando os Dias de Atraso

Comparando duas datas

Além de informar que o empréstimo está atrasado, a API pode calcular há quantos dias o prazo venceu.

No arquivo Emprestimo.java:

- adicione a importação do java.time.temporal.ChronoUnit;
- acrescente o método getDiasAtraso()

O método retorna 0 quando:

- o livro já foi devolvido;
- não existe uma data prevista;
- a previsão ainda não venceu.

Quando o empréstimo está atrasado, o método:

- ChronoUnit.DAYS.between(dataInicial, dataFinal)

calcula a quantidade de dias entre a previsão de devolução e a data atual.

Emprestimo.java

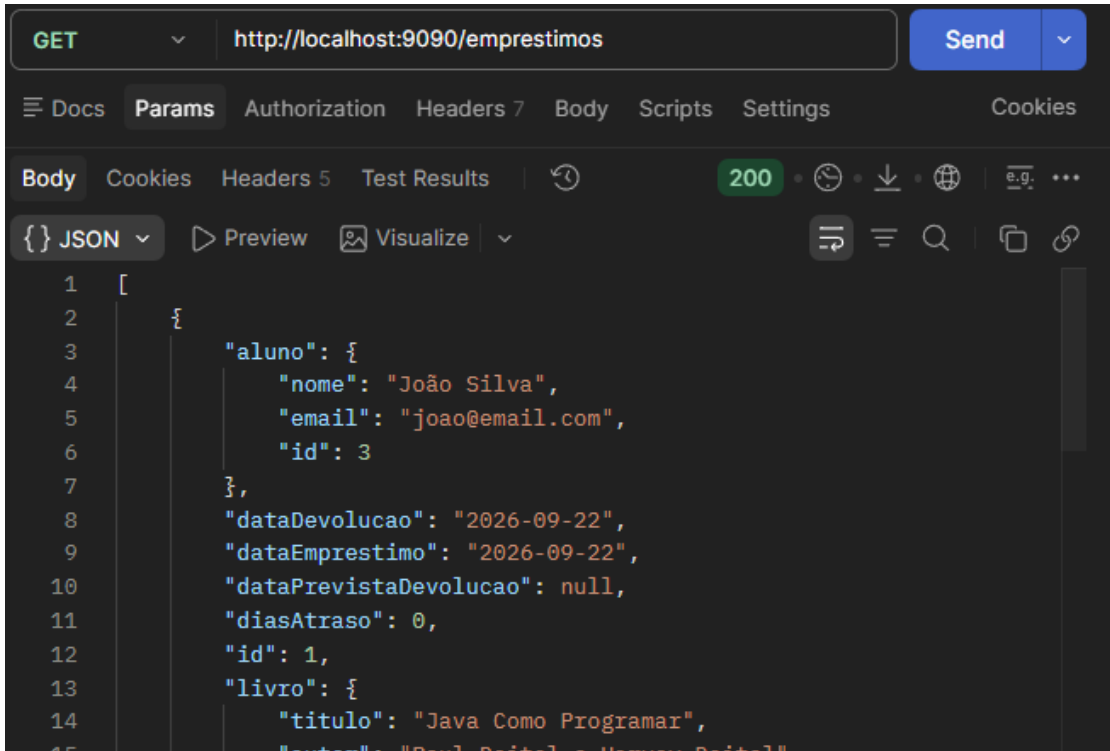
```java
public long getDiasAtraso() {

    if (dataDevolucao != null
            || dataPrevistaDevolucao == null
            || !dataPrevistaDevolucao.isBefore(
                    LocalDate.now())) {

        return 0;
    }

    return ChronoUnit.DAYS.between(
            dataPrevistaDevolucao,
            LocalDate.now());
}
```

No Postman



Por que o Java retorna long?

Foi utilizado long porque o método ChronoUnit.DAYS.between() já retorna um valor do tipo long.

Saiba Mais!



O tipo long suporta números inteiros muito maiores que int:

- int suporta 2,1 bilhões
- Long suporta 9 quintilhões

Fique ON



Na prática, nenhum empréstimo ficará bilhões de dias atrasado. Porém, ChronoUnit.DAYS.between() foi desenvolvido para calcular intervalos entre datas muito distantes e, por isso, utiliza long.



12.8

Multas

Calculando valor de multas

Firjan SENAI

Calculando a Multa

Utilizando os dias de atraso

A biblioteca cobrará uma multa de R$ 2,00 por dia de atraso. O valor será calculado pela aplicação e não será armazenado no banco de dados.

No arquivo Emprestimo.java:

- adicione a importação do java.math.BigDecimal;
- crie uma constante com o valor diário
- Adicione o método getter

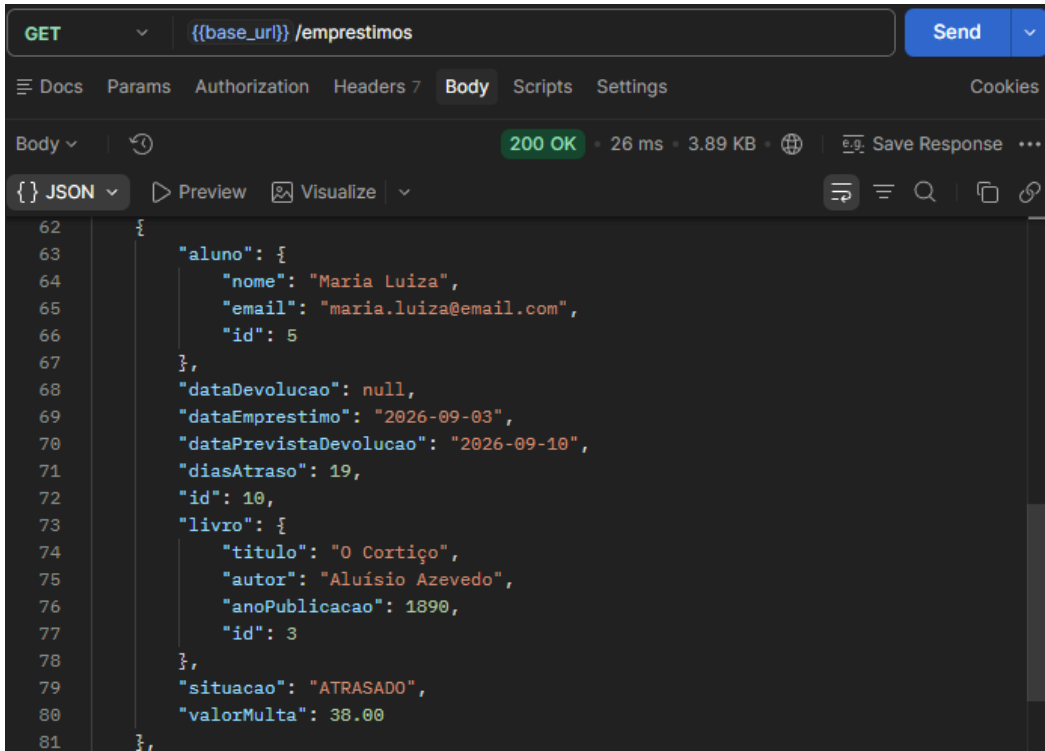
Emprestimo.java

```java
import java.math.BigDecimal;
```

```java
private LocalDate dataEmprestimo;
private LocalDate dataDevolucao;
private LocalDate dataPrevistaDevolucao;
private static final BigDecimal MULTA_POR_DIA = new BigDecimal("2.00");
```

```java
public BigDecimal getValorMulta() {

    return MULTA_POR_DIA.multiply(
            BigDecimal.valueOf(getDiasAtraso()));
}
```



Por que usar BigDecimal para a multa?

O BigDecimal não é usado para calcular a quantidade de dias. Os dias continuam sendo um long. Ele é utilizado para representar valores monetários com maior precisão, evitando pequenas imprecisões que podem ocorrer com double.

Atenção



BigDecimal já faz parte do Java (java.math.BigDecimal), não sendo uma biblioteca externa. Para uma atividade simples, double também funcionaria, por exemplo: getDiasAtraso() * 2.00. Porém, quando trabalhamos com dinheiro, o uso de BigDecimal é uma prática mais adequada, pois mantém maior precisão nos cálculos decimais.

double = mais simples | BigDecimal = mais adequado para dinheiro

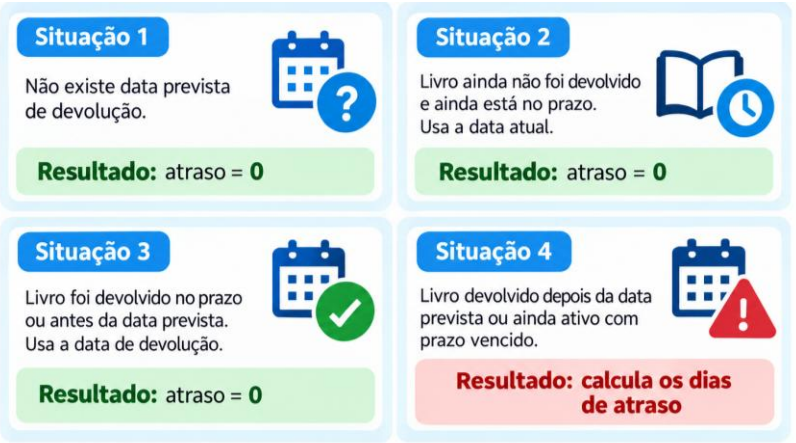
Preservando o histórico da multa

Até aqui, a API já calcula os dias de atraso e o valor da multa. Agora, observe que quando um empréstimo atrasado é marcado como devolvido o valor da multa some do histórico. Use a Situação de Aprendizagem para corrigir esse comportamento e verificar o resultado no Postman.

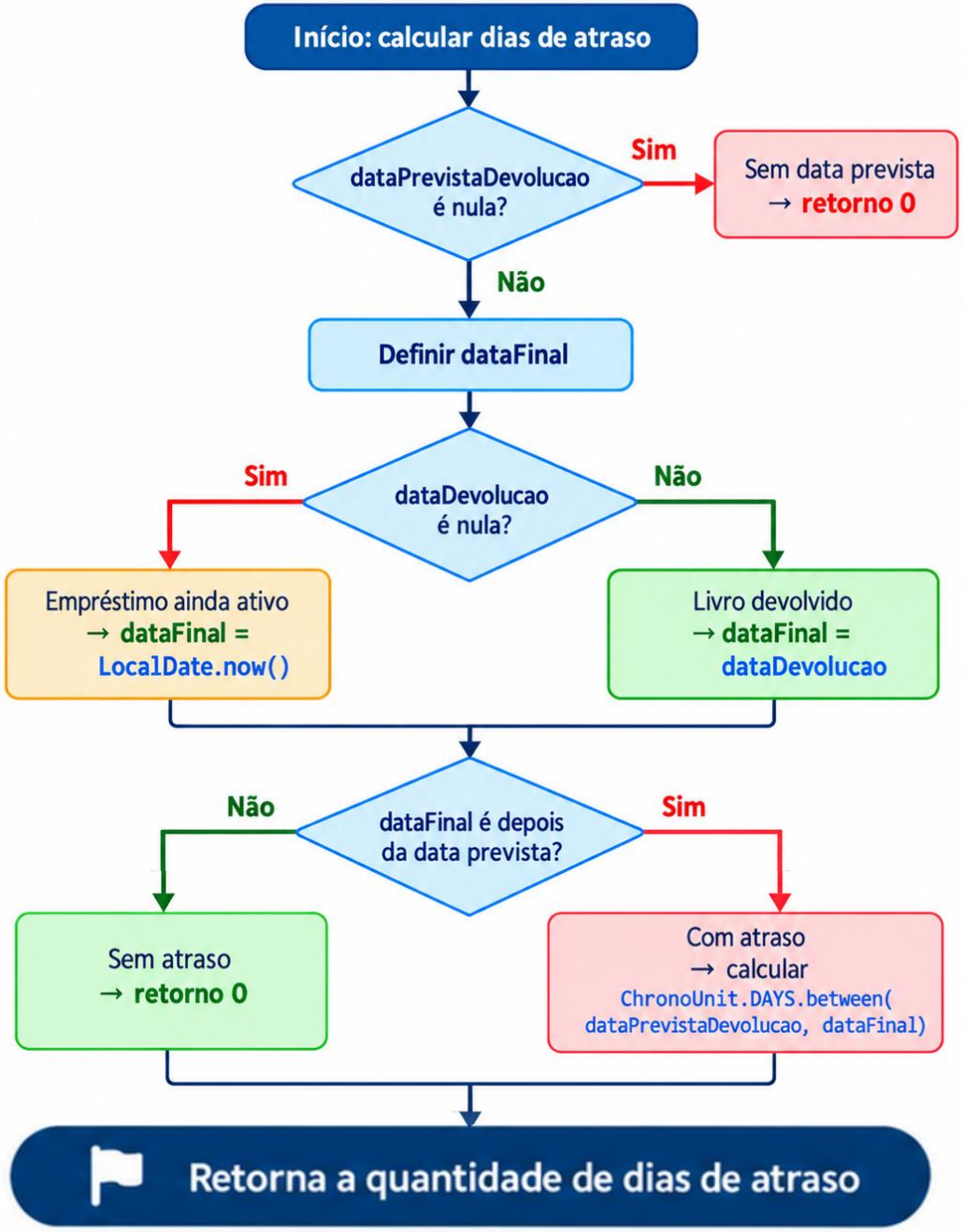
Desafio



Um livro devolvido com atraso aparece como DEVOLVIDO, mas os dias de atraso e a multa somem do JSON. Sua missão é ajustar esse comportamento sem alterar o cálculo dos empréstimos ainda ativos. Consulte a Situação de Aprendizagem "Desenvolvimento de Sistemas - Histórico de multa por atraso" para conhecer as condições do desafio e os resultados esperados.



Como o método `getDiasAtraso()` decide o atraso



Emprestimo.java

```java
public long getDiasAtraso() {

    if (dataPrevistaDevolucao == null) {
        return 0;
    }

    LocalDate dataFinal;

    if (dataDevolucao == null) {
        dataFinal = LocalDate.now();
    } else {
        dataFinal = dataDevolucao;
    }

    if (!dataFinal.isAfter(dataPrevistaDevolucao)) {
        return 0;
    }

    return ChronoUnit.DAYS.between(
            dataPrevistaDevolucao,
            dataFinal);
}
```

Firjan SENAI

Atraso Após a Devolução

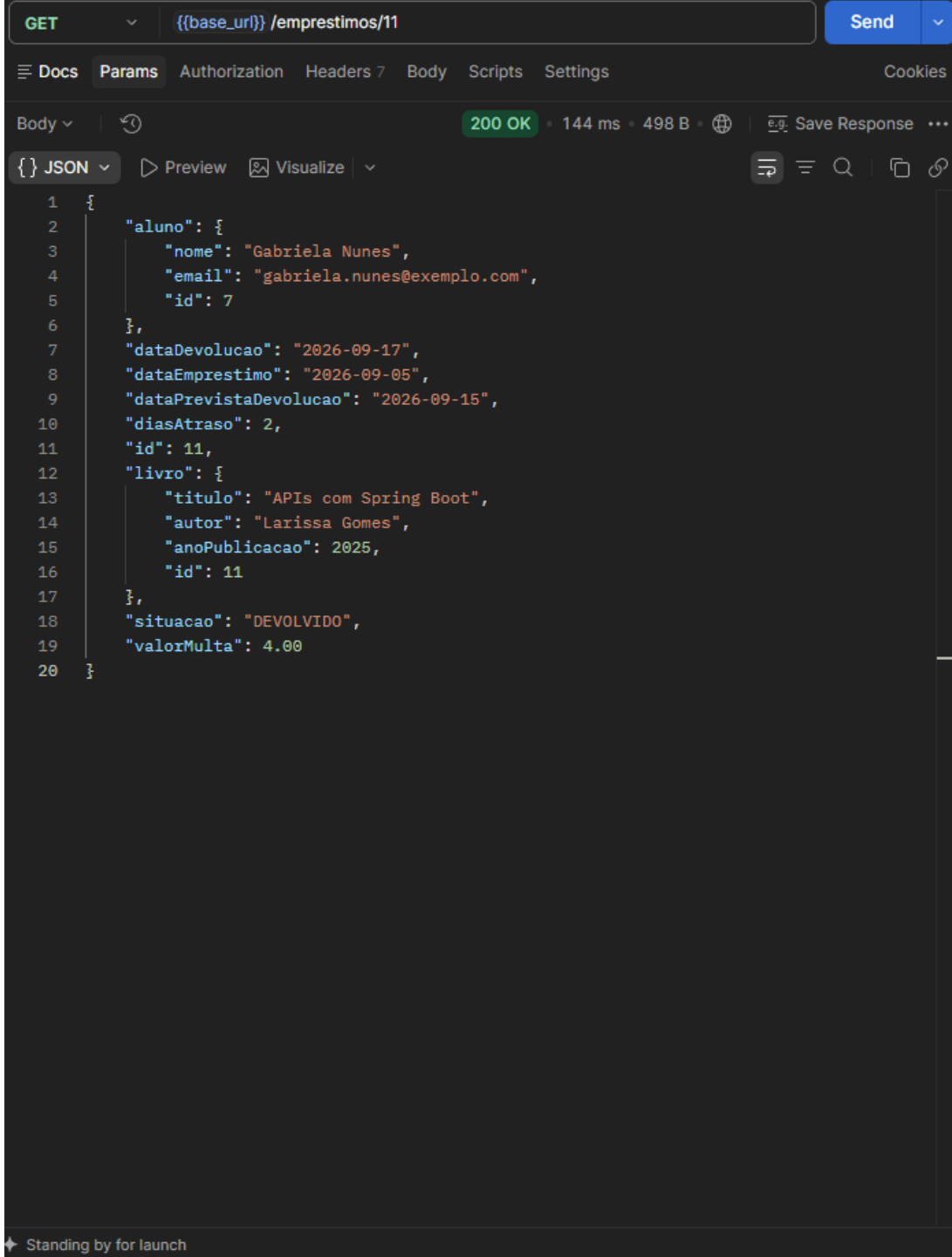
Preservando o histórico da multa

Para testar especificamente o último caso — manter os dias de atraso e a multa após a devolução — utilize o empréstimo de ID 11.

Teste no postman:

- GET http://localhost:8080/emprestimos/11
- GET http://localhost:8080/emprestimos/10

O ponto principal é perceber que, mesmo com a situação DEVOLVIDO, os valores não retornam para zero. O método utiliza a dataDevolucao como data final do cálculo e preserva o histórico do atraso.



Requisitos do Sistema - O que o sistema faz e quais regras deve respeitar.

Durante o desenvolvimento da API, implementamos funcionalidades solicitadas pelo usuário e condições que controlam como essas funcionalidades devem funcionar.

Resumo



Requisitos funcionais

Representam ações que o sistema deve disponibilizar:

- cadastrar um empréstimo;
- consultar empréstimos;
- buscar empréstimos por aluno;
- listar empréstimos ativos e atrasados;
- registrar uma devolução;
- renovar a data prevista;
- calcular dias de atraso e multa.

Regras de negócio

Definem as condições que devem ser respeitadas:

- um livro não pode possuir dois empréstimos ativos;
- a previsão de devolução deve estar no futuro;
- um empréstimo devolvido não pode ser renovado;
- a nova previsão deve ser posterior ao prazo atual;
- somente empréstimos não devolvidos aparecem como atrasados;
- a multa corresponde a R$ 2,00 por dia de atraso;
- o atraso histórico deve permanecer após a devolução.

Firjan SENAI